Programming Fundamentals 2:
Exam I Study Guide

Exam Information

Breakdown:
150 total points

- 4 Free Response Questions, 30 pts per FRQ
- 10 Multiple Choice Questions, 3 pts per MCQ

FRQ Topics:
1- Classes: Default & Parameterized Constructor, Setters and Getters
2- Pass by Copy, by Reference, or by Pointer.
3- Dynamic Memory Allocation and Big Three
4- Template Class and Operator Overloading

MCQ Topics:
Anything from Module 1 to Module 6, up to Linked Lists

Guided Topics for Review

Class Basics

- Default and Parameterized Constructor
    - What is the difference between the default and parameterized constructor?
    - How many default constructors can you make? How many parameterized?
    - How could you combine the default and parameterized constructor into one declaration/definition?
        - Try writing out the syntax for it!
- Setters and Getters
    - What is a setter function? And a getter function?
    - Why would you write a setter and getter function for a private variable?
    - Try creating a class with a setter and getter for at least 2 separate variables.

- Access Specifiers
    - What is the difference between private vs public within a class?
    - What is the difference between a class and a struct?
    - Where should variables go, private or public? And what about functions?

References and Pointers

- Syntax Labelling
  - Try labelling each of the following:
    1. `int& myThing = yourThing;`
    2. `int& favNumber = 37;`
    3. `int myValue = *someValue;`
    4. `int number = 10;`
    5. `int* bigAns = 42;`
    6. `int* myObj = &otherObj;`
  - With one of these labels (one-to-one):
    a. Normal variable assignment
    b. Reference assignment
    c. Pointer assignment
    d. Pointer dereferencing
    e. Invalid reference assignment
    f. Invalid pointer assignment
- Passing Into Functions
  - How do you pass by copy into a function?
    - What are the limitations of passing by copy?
  - How do you pass by reference into a function?
    - What benefits does this have over passing by copy?
      - Space efficiency?
      - Direct modification?
    - What are the risks?
    - Can this work if you pass it an immediate/literal value?
      - Ex: `fooFunc(5), barFunc("Howdy!")`
  - How do you pass by pointer into a function?
    - What exactly are you passing into the argument?
    - What happens if you modify the pointer variable value? Does it change the original pointer?
    - What happens if you change the value being pointed to? Does it change the value the original pointer is pointing to?

Operator Overloading

- Function Overloading
  - What does it mean to overload a function?
  - How do you overload a function? Try overloading a simple function called “printThing” with an int argument or a string argument and print out its provided value.
- Common Operators
  - What is the difference between the + and += operators?
  - What is the difference between the += and >= operators?
  - Match each of the following operator collections:
    1. `+, -, /, *`
    2. `+=, -=, /=, *=, =`
    3. `==, >=, <=, >, <`
  - With the following labels (one-to-one) :
    a. Comparison operators
    b. Assignment operators
    c. Arithmetic operators
- Operator Overload Syntax
  - List out the return types and expected arguments of the following:
    - `operator+`
    - `operator-`
    - `operator-=`
    - `operator=`
    - `operator!=`
    - `operator<`
  - Which of the above operators could have void return type? Which shouldn’t? Are there better return type choices rather than void?
  - Which operators change the calling object (this)?
  - Could you use const in these statements? For what?
  - In the statement **`myObj C = A + B`**, assuming A and B are initialized instances of myObj:
    - What overloads get called, and in what order?
    - What are their expected return types and arguments here?

Dynamic Memory and Big Three

- Comparison with Static Memory
  - **Sta**tic memory is stored on the **Sta**ck, Dynamic is stored on the Heap
  - Static is known at compile time, Dynamic is known at run time
  - Static is automatically deleted, Dynamic has to be freed/deleted manually (such as in the destructor)
- Dynamic Memory Syntax
  - What does the **new** keyword do?
    - How do you create a dynamic object?
    - What about a dynamic array?
    - What type does **new** return?
  - What does the **delete** keyword do?
    - When should you use **delete** vs **delete[]**?
- Big Three
  - Here is a code snippet that makes use of all three important functions in the Big Three:

```
1 main() {
2     myDynClass veteran;
3     myDynClass middleman;
4     middleman = veteran;
5     myDynClass noobie = veteran;
6 }
```

  - Answer the following for the Big Three:
    - How do you define the destructor for a class?
    - Why do we need to redefine it when using dynamic memory?
    - When is this called in the above code snippet? And for which instances?
  - Big Three:
    - Copy Constructor
    - Copy Assignment Operator
    - Destructor

Template Classes and Functions

- Template Syntax
    - How do you declare a template function with a generic type T?
    - What about a template class?
    - Can you declare and define template functions and classes in different files?
    - What can you use T as in your template class/function?
    - How could you have multiple generic types? Does it have to be called T?
- Template Classes
    - Should you split template classes into separate .h and .cpp files?
        - What file types does this splitting work for?
        - What error do you get if done incorrectly?
    - Can you have more than one template in a template class?
        - Is this the correct way to define a template with 2 variables? Template <Typename T, Typename K>
        - What would the object look like when you create a variable from a template with 2 variables?
    - Can you access a template within a subclass if the template was defined in an external class?
        - What about vice versa?
        - Where would this be useful for?
    - Did you place templates on your cheat sheet?
        -